

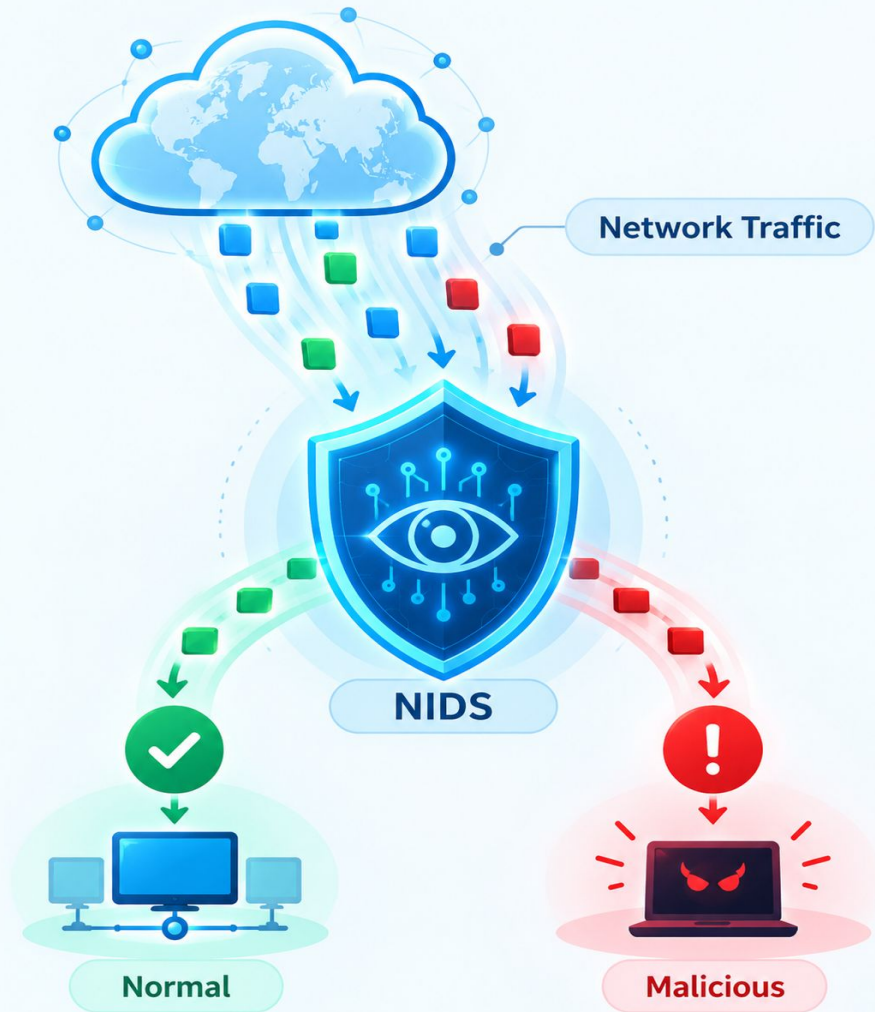
Evaluating and Improving Adversarial Robustness of Machine Learning-Based Network Intrusion Detectors

By Luis Hernandez

Can hackers trick ML security by changing how network traffic looks?

Based on Han et al., IEEE JSAC 2021.





What is Network Intrusion Detection System (NIDS) and what does it do?

A **Network Intrusion Detection System** watches network traffic and tries to identify suspicious or malicious activity.

NIDS generally fall into two categories:

Signature-Based NIDS

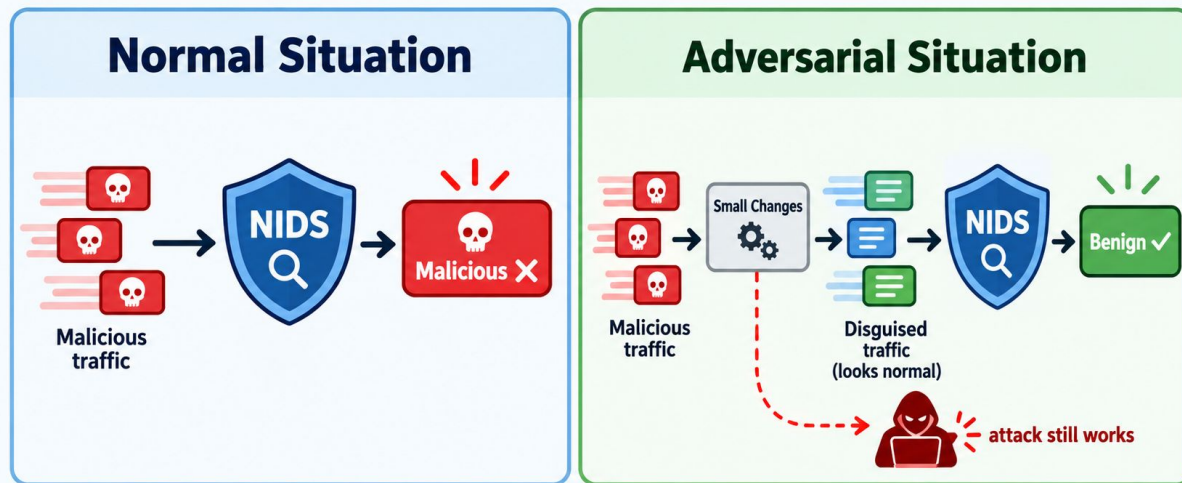
- Looks for known attack patterns
- Works well on attacks it already recognizes
- Struggles with new or modified attacks

ML / Anomaly-Based NIDS

- Learns patterns from network traffic
- Detects behavior that looks unusual
- Can potentially detect attacks it has never seen before

The Problem: Machine Learning Can Be Fooled

A small, carefully chosen change to an input can cause a machine-learning model to make the **wrong decision**. The paper focuses on **evasion attacks**, where malicious network traffic is changed so the NIDS classifies it as benign.





Proposed Attack: GAN + PSO

Two steps, find where to go, then figure out how to get there.

Generative Adversarial Network (GAN) to Find the Target

The GAN takes malicious traffic features and learns to generate an **adversarial feature target** that:

- Looks more benign to the ML detector
- Stays relatively close to the original malicious features
- Does **not require knowledge of the target classifier**

In simple terms:

GAN asks: “What should this attack look like to fool the detector?”

Particle Swarm Optimization (PSO) Modify the Real Traffic

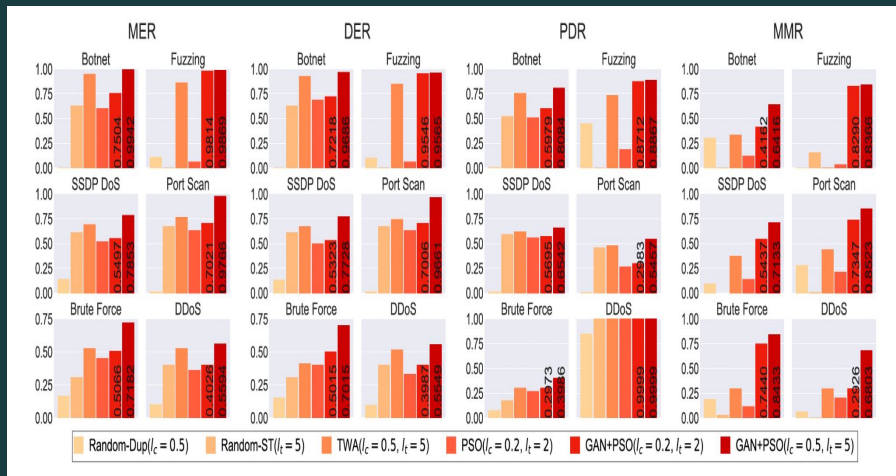
PSO then searches for actual network-traffic changes that move the traffic toward the GAN's target.

It can adjust things such as:

- Packet timing
- Added crafted packets
- Protocol-layer characteristics
- Payload size of crafted packets

The search continues until it finds traffic whose extracted features are close to the adversarial target.

Did the Attack Actually Work?



Higher = more successful attack

The authors attacked **Kitsune**, a machine-learning-based NIDS, using six kinds of malicious traffic:

- Botnet
- Fuzzing
- SSDP DoS
- Port Scan
- Brute Force
- DDoS.

MER – Malicious Traffic Evasion Rate

How much of the original malicious traffic successfully avoids detection.

DER – Detection Evasion Rate

Like MER, but also considers whether the additional crafted traffic itself gets detected.

PDR – Probability Decline Rate

How much the detector's confidence that traffic is malicious decreases.

MMR – Mimicry Rate

How closely the modified malicious features move toward the desired adversarial features.

They compared their proposed **GAN + PSO attack** against:

- Random traffic mutations
- A prior **traffic-space white-box attack (TWA)**
- PSO without adversarial features

The point was to see whether their method could make malicious traffic evade detection while operating under realistic mutation limits.

What Did the Attack Reveal?

Finding 1

The ML model matters

Robustness varied significantly between classifiers. In the Botnet tests, traditional ML methods were generally more robust than the deep-learning models. **KitNET had strong normal detection performance but also suffered high evasion rates**, while **Isolation Forest was relatively more robust overall**.

Finding 2

The features matter

NIDS using **flow-based features** were **slightly more robust** than systems using packet-based features.

This is important because the attack works by manipulating packet characteristics, which then influence the features used by the detector.

Finding 3

Hiding the system is not enough

When the attacker knew **0% of the exact features** used by Kitsune, the black-box attack still significantly reduced the detector's malicious probability.

The authors found that the PDR degradation compared with full feature knowledge remained within about **20%**.

Main Statement:

A NIDS does not become secure simply because the attacker does not know how it works.

So if certain features are creating the vulnerability, can we remove those vulnerable features?

Their proposed defense – Adversarial Feature Reduction (AFR).

Core idea

Remove the most vulnerable features instead of letting the attacker keep exploiting them.

How AFR works

1. Simulate the adversarial attack

Run the proposed attack against the NIDS.

2. Measure each feature robustness

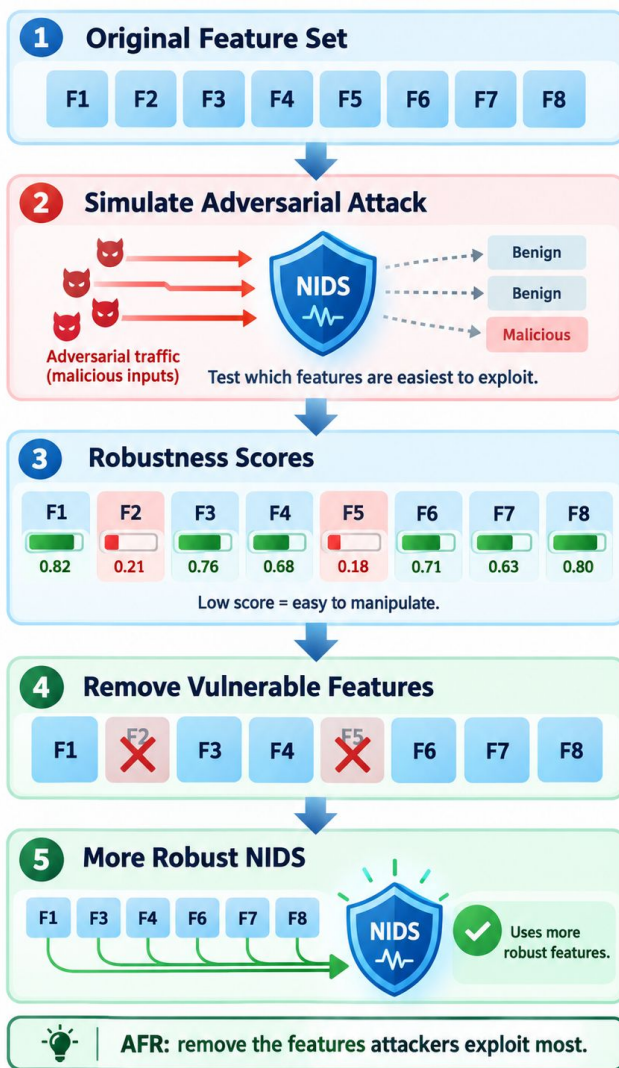
The authors calculate how much each feature dimension moves toward the adversarial target during the attack.

3. Identify weak features

Features that are easily manipulated receive lower robustness scores.

4. Remove the weakest dimensions

The NIDS is retrained using the remaining, more robust features.



Did AFR Actually Work?

The paper compares three defenses:

- **AT – Adversarial Training**
- **FS – Feature Selection**
- **AFR – Adversarial Feature Reduction**

What the figure shows

The bars measure how much each defense reduces:

- MER – malicious traffic evasion
- PDR – malicious probability decline
- MMR – movement toward adversarial features

AFR generally performed better than adversarial training and standard feature selection.

Big Takeaway:

Removing the features most vulnerable to attack made the NIDS harder to fool without greatly hurting normal detection performance.

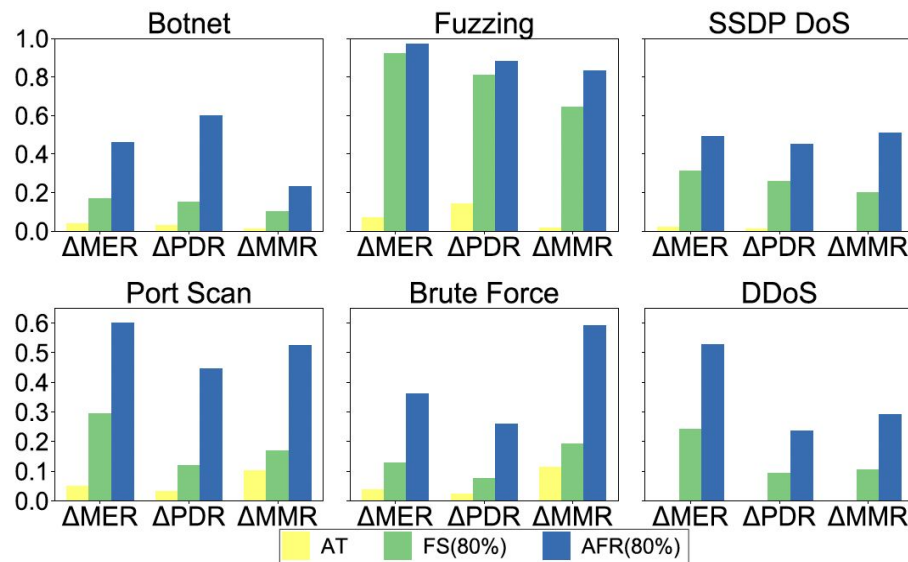


Fig. 6. The defense performance (*higher is better*).

How AFR Works: The Full Defense Pipeline

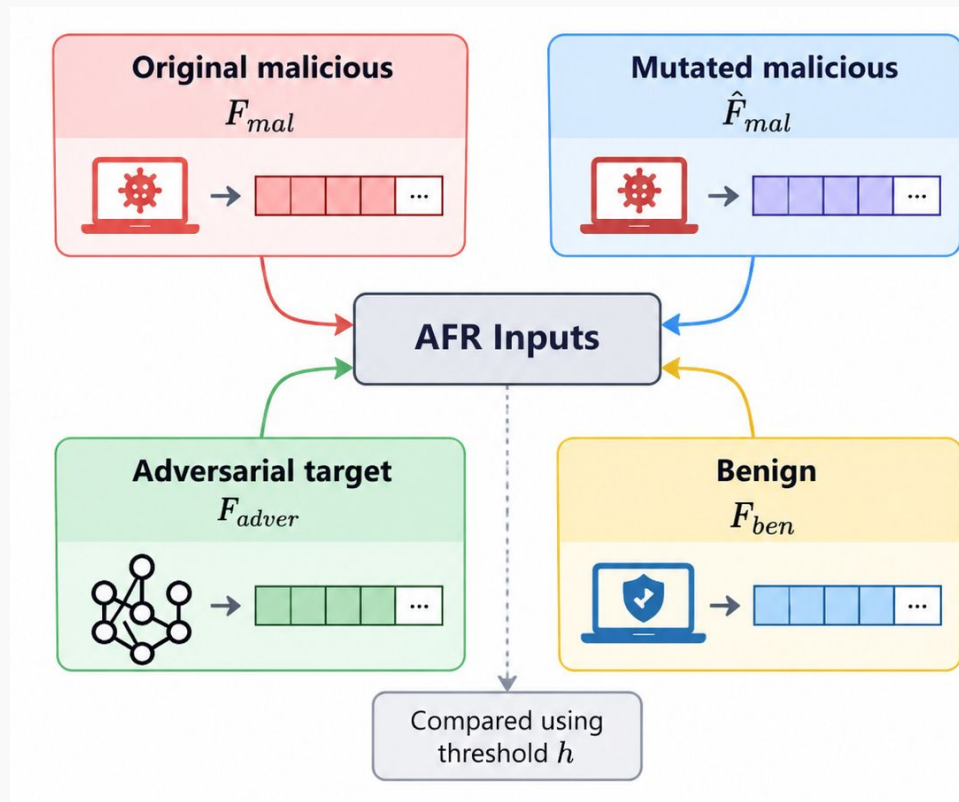


The paper's defense follows this overall process: it proactively runs the adversarial attack, measures how each feature behaves during successful evasions, assigns robustness scores, removes the most vulnerable feature dimensions, and then evaluates the NIDS using the reduced feature set.

Step 1 Generate the data AFR needs

The paper's AFR algorithm starts with four feature sets plus the NIDS anomaly threshold h .

1. **Original Malicious Features**
 - a. Features extracted from the original malicious traffic before modification.
2. **Mutated Malicious Features**
 - a. Features extracted from the traffic after GAN + PSO modifies it.
3. **Adversarial Target Features**
 - a. These are the adversarial features generated by the GAN.
4. **Benign Features**
 - a. These are features extracted from normal network traffic. They provide the reference for what normal/benign traffic looks like.



Step 2 Measure How Much Each Feature Mimics the Adversarial Target

MMR = Malicious Features Mimicry Rate

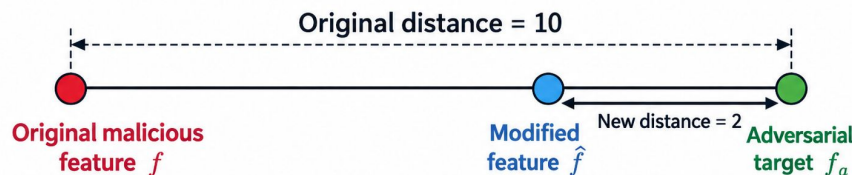
The paper introduces **MMR** to measure how much the modified malicious features move toward the adversarial target during the attack.

The paper defines it as:

$$MMR = 1 - \mathbb{E} \left[\frac{\mathcal{L}(\hat{f}, f_a)}{\mathcal{L}(f, f_a)} \right]$$

- f = original malicious feature vector
- $\hat{f}$ = mutated malicious feature vector
- f_a = adversarial target feature vector
- $\mathcal{L}(\cdot, \cdot)$ = distance between two feature vectors

What does MMR mean?



$$MMR = 1 - \frac{2}{10} = 0.8$$

The modified feature moved 80% of the way toward the adversarial target.

Higher MMR = feature changed more toward the target.

MMR tells AFR which features the attacker successfully moved toward the adversarial region.

Step 3 The AFR Robustness-Score Algorithm

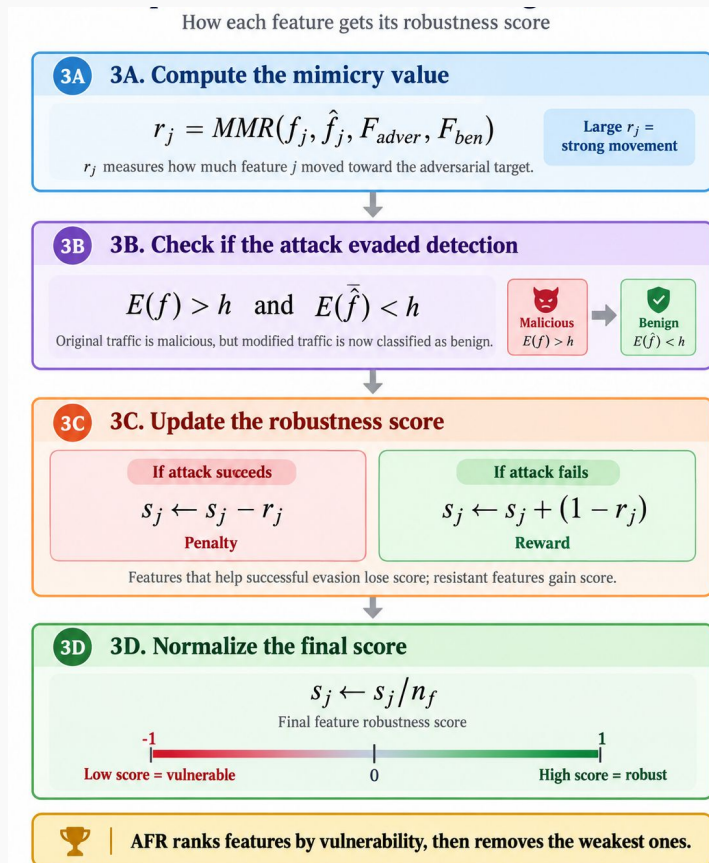
AFR gives every feature dimension a score based on how easy it was to exploit.

Core idea

For each feature dimension j , AFR:

1. Measures how much that feature moved toward the adversarial target
2. Checks whether the attack actually **evaded detection**
3. Updates the feature's robustness score accordingly

At this stage, AFR is assigning a robustness score to each feature. First it measures how much that feature moved toward the adversarial target. Then it checks whether the whole attack actually fooled the detector. If the attack succeeded, the feature is penalized. If the attack failed, the feature is rewarded. After doing this across many samples, AFR ends up with a vulnerability ranking for all features.



Step 4 Remove the Weakest Features

After Step 3, every feature dimension has an adversarial robustness score.

The paper's idea is simple:

Low robustness score = feature is easier for the attacker to exploit.

What did the paper actually do?

In the defense experiment, the authors configured AFR to **retain 80% of the feature dimensions**.

So conceptually:

100% features → rank by robustness → remove weakest 20% → 80% retained

They used the same 80% retention level for conventional feature selection so the defenses could be compared more fairly.

Why should this help?

The paper argues that high-dimensional feature sets give attackers more opportunities to exploit vulnerable dimensions. Removing the dimensions that the attack manipulates most successfully reduces those opportunities.



AFR removes the features that are **easiest** for the attacker to exploit.

Limitations of the Proposed Method

Non-payload NIDS only

The proposed attack is designed for NIDS that rely on **non-payload features** such as packet timing, size, and counts. By itself, it does not address detectors that also inspect packet contents.

The attack is currently offline

The authors generate the modified traffic first and then replay it. It was not implemented as a fully real-time online attack.

Background traffic may interfere

Real networks contain traffic the attacker does not control. That traffic can alter some aggregated features and potentially affect the crafted attack, although the authors found the impact limited in their experiments.

Takeaway

The method demonstrated a serious weakness, but it was not designed as the strongest possible attack or a complete model of every real network.

Future Work

The paper points to several clear directions for improving both the attack framework and the defense.

1. Extend beyond non-payload features

The current method focuses on NIDS that do not inspect packet payloads. The authors suggest combining their approach with payload-focused evasion methods so the attack could also target systems that inspect packet contents.

2. Move from offline to online attacks

The attack is currently generated offline and then replayed. A future version could perform traffic mutation in real time while the malicious activity is happening.

3. Improve GAN + PSO optimization

The authors say they mainly used default or recommended settings. Future work could tune:

- PSO parameters
- GAN parameters
- number of adversarial targets
- search strategy

to potentially improve evasion effectiveness.

4. Explore better distance functions

The paper uses **Euclidean distance** to measure similarity in feature space. The authors explicitly suggest testing other distance metrics.

5. Design more robust NIDS from the start

The authors also argue that future NIDS design should consider not just normal detection performance, but also **adversarial robustness when selecting features**.

Acknowledgements and References

- [1]** D. Han, Z. Wang, Y. Zhong, W. Chen, J. Yang, S. Lu, X. Shi, and X. Yin,
“Evaluating and Improving Adversarial Robustness of Machine Learning-Based
Network Intrusion Detectors,”
IEEE Journal on Selected Areas in Communications, vol. 39, no. 8, pp. 2632–2647,
Aug. 2021.
DOI: **10.1109/JSAC.2021.3087242**.

Implementation provided by the authors:

github.com/dongtsi/TrafficManipulator

ChatGPT was used to generate illustrative visuals and assist
with slide organization.

Questions?